

Přehled jazyků, rekapitulace, závěr

IB111 ZÁKLADY PROGRAMOVÁNÍ

Tomáš Vojnar (na základě slajdů Nikoly Beneše)

12. prosince 2024

Přehled programovacích jazyků

- Rychlý pohled na historii.
- Klasifikace programovacích jazyků.

Shrnutí předmětu

- Co jste se měli naučit.
- Co dál.

První programátor(ka)

Ada, hraběnka z Lovelace.

Napsala první program pro počítač,
který nikdy nebyl sestrojen
(analytický stroj Charlese Babbage).

[en.wikipedia.org/wiki/
Analytical_Engine](https://en.wikipedia.org/wiki/Analytical_Engine)



On two occasions I have been asked, — “Pray, Mr. Babbage, if you put into the machine wrong figures, will the right answers come out?”

In one case a member of the Upper, and in the other a member of the Lower, House put this question. I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question.

Passages from the Life of a Philosopher (1864),
ch. 5 “Difference Engine No. 1”

Historie

- První počítač (nikdy nesestrojen): 1837 Charles Babbage.
 - Děrné štítky (používány pro tkací stroje od pol. 18. století).
- Teoretické základy programování: 30.–40. léta.
 - Alonzo Church, Alan Turing, ...
- První počítače: 40. léta.
 - Strojový kód, první programovací jazyky – zejména assembler.
- Počátek programovacích jazyků vyšší úrovně: 50.–60. léta.
 - FORTRAN, ALGOL, LISP, COBOL.
- Další vývoj: 70.–80. léta.
 - Strukturované (bez goto), objektově-orientované programování.
 - Rozvoj jiných druhů programovacích jazyků (logické, funkcionální).
- Moderní programovací jazyky:
 - stále vznikají nové jazyky, staré jazyky se vyvíjejí a mění.
 - <https://www.youtube.com/watch?v=ZTPrbAKmcd0>.

Podle míry abstrakce

- Jazyky vyšší/nížší úrovně.

Podle paradigmatu

- Imperativní/deklarativní.

Další druhy

- Skriptovací.
- Objektově-orientované.
- Paralelní.

Většina (moderních) jazyků kombinuje více přístupů.

Imperativní jazyky

- Program je posloupnost příkazů, která popisuje, *jak* má být výpočet proveden.
- Popisuje, jak se má postupně měnit stav výpočtu.

Imperativní jazyky

- Program je posloupnost příkazů, která popisuje, *jak* má být výpočet proveden.
- Popisuje, jak se má postupně měnit stav výpočtu.

Deklarativní jazyky

- Program je popis toho, *co* má být výsledkem.
- **Logické programování**: Prolog, Datalog, ASP-Core-2, ...
 - Program tvoří logické formule (fakta, odvozovací pravidla).
- **Funkcionální programování** (Haskell, Ocaml, Lisp, ...)
 - Program je popsán pomocí aplikací a skládání funkcí.
 - Funkce mohou být přiřazovány, předávány/vraceny do/z funkcí.
 - Funkce nemají vedlejší efekty – v čistém funkc. programování.
- Velmi běžnou konstrukcí je rekurse.

Paradigmata programovacích jazyků

Imperativní jazyky

- Program je posloupnost příkazů, která popisuje, *jak* má být výpočet proveden.
- Popisuje, jak se má postupně měnit stav výpočtu.

Deklarativní jazyky

- Program je popis toho, *co* má být výsledkem.
- **Logické programování**: Prolog, Datalog, ASP-Core-2, ...
 - Program tvoří logické formule (fakta, odvozovací pravidla).
- **Funkcionální programování** (Haskell, Ocaml, Lisp, ...)
 - Program je popsán pomocí aplikací a skládání funkcí.
 - Funkce mohou být přiřazovány, předávány/vraceny do/z funkcí.
 - Funkce nemají vedlejší efekty – v čistém funkc. programování.
- Velmi běžnou konstrukcí je rekurse.

Moderní jazyky kombinují více přístupů:

- funkcionální prvky/paralelismus/... v C++, Javě, Pythonu, ...

Skriptovací jazyky

- Užitečné (zejména) pro jednoduché programy, tzv. *skripty*.
- Usnadnění zdlouhavé manuální činnosti.
- Obvykle interpretované, často ze zdrojového kódu (AST).
- Provádí se obvykle od prvního řádku.
- Bash, Perl, Python, ...

Skriptovací jazyky

- Užitečné (zejména) pro jednoduché programy, tzv. *skripty*.
- Usnadnění zdlouhavé manuální činnosti.
- Obvykle interpretované, často ze zdrojového kódu (AST).
- Provádí se obvykle od prvního řádku.
- Bash, Perl, Python, ...

Objektově-orientované jazyky

- Objektový návrh programů: třídy, metody, dědičnost, ...
- Většina moderních jazyků obsahuje nějaké prvky OOP.
- C++, Java, Python, Objective C, C#, ...

Skriptovací jazyky

- Užitečné (zejména) pro jednoduché programy, tzv. *skripty*.
- Usnadnění zdlouhavé manuální činnosti.
- Obvykle interpretované, často ze zdrojového kódu (AST).
- Provádí se obvykle od prvního řádku.
- Bash, Perl, Python, ...

Objektově-orientované jazyky

- Objektový návrh programů: třídy, metody, dědičnost, ...
- Většina moderních jazyků obsahuje nějaké prvky OOP.
- C++, Java, Python, Objective C, C#, ...

Paralelní výpočty

- Počítání na více procesorech/více počítačích zároveň.
- Získává na popularitě (procesory už moc rychlejší nebudou).
- Podpora dostupná přímo v moderních jazycích.

Jaký jazyk používat?

Programovacích jazyků je mnoho. Který si mám vybrat?

Programovacích jazyků je mnoho. Který si mám vybrat?

- Špatně položená otázka.
- Programovací jazyky jsou **nástroje**.
- Není žádný jeden jazyk ideálně vhodný pro všechno.
- Záleží na tom, co řešíme.
- Může být výhodné kombinovat více jazyků/přístupů.

Programovacích jazyků je mnoho. Který si mám vybrat?

- Špatně položená otázka.
- Programovací jazyky jsou **nástroje**.
- Není žádný jeden jazyk ideálně vhodný pro všechno.
- Záleží na tom, co řešíme.
- Může být výhodné kombinovat více jazyků/přístupů.

Doporučení

- Nefixujte se na jeden jazyk.
- Různé jazyky s sebou nesou i různé způsoby přemýšlení.
 - Je dobré *rozumět principům*, na kterých jazyky stojí.
 - Ideálně včetně principů dalších souvisejících technologií a teorií.

Programovacích jazyků je mnoho. Který si mám vybrat?

- Špatně položená otázka.
- Programovací jazyky jsou **nástroje**.
- Není žádný jeden jazyk ideálně vhodný pro všechno.
- Záleží na tom, co řešíme.
- Může být výhodné kombinovat více jazyků/přístupů.

Doporučení

- Nefixujte se na jeden jazyk.
- Různé jazyky s sebou nesou i různé způsoby přemýšlení.
 - Je dobré *rozumět principům*, na kterých jazyky stojí.
 - Ideálně včetně principů dalších souvisejících technologií a teorií.

„Kolik programovacích jazyků umíš, tolikrát jsi programátorem.“

Co jste se měli naučit?

- Základy programování:
 - jednoduché výpočty, zpracování dat;
 - alespoň trochu rozumět kultuře programování;
 - dostatečné k tomu, abyste se mohli naučit jiný jazyk.
- Základy algoritmizace a složitosti.
- Základy Pythonu (Python jako takový nebyl cílem).

Základní programovací konstrukce

Příkaz vs. výraz

Podprogramy (funkce)

Čisté funkce, predikáty, vedlejší efekty

Datové typy, složené datové typy, anotace

Proměnné, objekty, vazby mezi nimi

Korektnost, vstupní a výstupní podmínky

Základy složitosti

Čitelnost kódu, dokumentace a komentáře

Základy algoritmického myšlení

- Princip půlení intervalu:
 - binární vyhledávání,
 - souvislost s logaritmem.
- Euklidův algoritmus pro výpočet největšího společného dělitele.
- Eratosthenovo síto pro výpočet prvočísel.
- Základní řadicí algoritmy: Select Sort, Insert Sort.

Základy algoritmického myšlení

- Princip půlení intervalu:
 - binární vyhledávání,
 - souvislost s logaritmem.
- Euklidův algoritmus pro výpočet největšího společného dělitele.
- Eratosthenovo síto pro výpočet prvočísel.
- Základní řadicí algoritmy: Select Sort, Insert Sort.

Abstraktní datové typy a datové struktury

- Seznam, zásobník, fronta, množina, slovník.
- Zřetěžené seznamy, dynamické pole.
- Stromy.

Vlastní datové typy

- Součtové a součinnové typy.
- Vlastní datové typy v Pythonu.
 - Třídy, metody, inicializace objektů, volání metod.

Vlastní datové typy

- Součtové a součinnové typy.
- Vlastní datové typy v Pythonu.
 - Třídy, metody, inicializace objektů, volání metod.

Interakce s prostředím

- Standardní vstup/výstup/chybový výstup.
- Základy práce s (textovými) soubory:
 - blok `with`,
 - režimy otevření souboru, čtení/zápis.
- Grafémy, kódové body a jednotky.
- Práce s parametry příkazového řádku.

Vlastní datové typy

- Součtové a součinnové typy.
- Vlastní datové typy v Pythonu.
 - Třídy, metody, inicializace objektů, volání metod.

Interakce s prostředím

- Standardní vstup/výstup/chybový výstup.
- Základy práce s (textovými) soubory:
 - blok `with`,
 - režimy otevření souboru, čtení/zápis.
- Grafémy, kódové body a jednotky.
- Práce s parametry příkazového řádku.

Základy správy paměti

- Od manuální po garbage collection.
- Různé typy kopií objektů.

Rekurze

- Způsob přemýšlení.
- Pravidla pro správnou rekurzi:
 - jednoduché vstupy řešíme přímo (báze),
 - rekurzivní volání problém zjednodušuje (o minimální krok).
- Koncová (tail) rekurze.
- Rekurze jako nástroj pro práci s rekurzivními datovými strukturami (např. stromy).
 - Práce s reprezentací výrazů (AST), jednoduchý interpret.
- Řešení problémů splnění omezení pomocí backtrackingu:
 - vhodná lokální volba,
 - rekurze + zastavení výpočtu ve chvíli, kdy už nesměřuje k cíli,
 - jedno/všechna/nej... řešení.

Algoritmy (směr k větší abstrakci)

- **IB002** Algoritmy a datové struktury I
 - Programátorské úlohy v Pythonu.
- Navazující **IV003** Algoritmy a datové struktury II.

Algoritmy (směr k větší abstrakci)

- **IB002** Algoritmy a datové struktury I
 - Programátorské úlohy v Pythonu.
- Navazující **IV003** Algoritmy a datové struktury II.

Programování hlouběji (směr nižší abstrakce, jak funguje počítač)

- **PB111** Principy nízkoúrovňového programování
 - Správa paměti, práce s ukazateli ... (jazyk C $\rightarrow$ strojový kód).

Algoritmy (směr k větší abstrakci)

- **IB002** Algoritmy a datové struktury I
 - Programátorské úlohy v Pythonu.
- Navazující **IV003** Algoritmy a datové struktury II.

Programování hlouběji (směr nižší abstrakce, jak funguje počítač)

- **PB111** Principy nízkoúrovňového programování
 - Správa paměti, práce s ukazateli ... (jazyk C → strojový kód).

Principy (moderních) jazyků

- **PB006** Principy programovacích jazyků a OOP.

Algoritmy (směr k větší abstrakci)

- **IB002** Algoritmy a datové struktury I
 - Programátorské úlohy v Pythonu.
- Navazující **IV003** Algoritmy a datové struktury II.

Programování hlouběji (směr nižší abstrakce, jak funguje počítač)

- **PB111** Principy nízkoúrovňového programování
 - Správa paměti, práce s ukazateli ... (jazyk C $\rightarrow$ strojový kód).

Principy (moderních) jazyků

- **PB006** Principy programovacích jazyků a OOP.

Jiná paradigmata

- **IB015** Neimperativní programování (a navazující předměty)
 - Haskell, Prolog.

Paralelní programování

- **IB109** Návrh a implementace paralelních systémů.
- Navazující **PV197** GPU Programming.
 - Používání grafických karet pro masivně paralelní výpočty

Paralelní programování

- **IB109** Návrh a implementace paralelních systémů.
- Navazující **PV197** GPU Programming.
 - Používání grafických karet pro masivně paralelní výpočty

Více o jazyce Python a jiných jazycích

- **PV288** Python + **PV248** Python Seminar.
- **PV178** Úvod do vývoje v C#/.NET (a navazující předměty).
- **PB161** Programování v C++ (a navazující **PV264/PV294**).
- **PB162** Programování v jazyce Java (a navazující předměty).
- **PV281** Programování v jazyce Rust.

Analýza, verifikace, testování

- **PB176** Základy kvality a správy kódu
- **PA013** Software Testing and Analysis
- **IA159** Formal Methods for Software Analysis

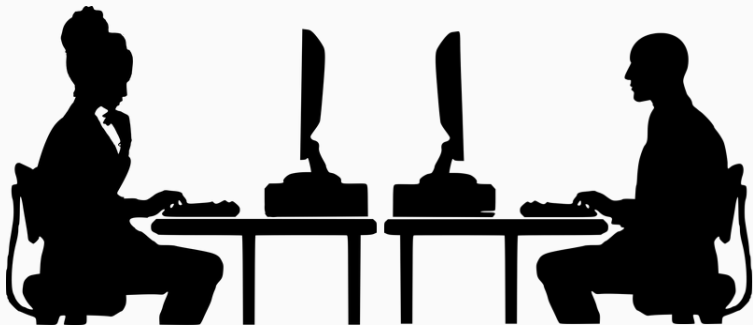
Analýza, verifikace, testování

- **PB176** Základy kvality a správy kódu
- **PA013** Software Testing and Analysis
- **IA159** Formal Methods for Software Analysis

Teorie programování

- Co je možné pomocí počítače řešit?
- Jak rychle je to možné řešit?
- Vyčíslitelnost a složitost,
 - učí se v rámci různých předmětů (**IB005**, **IB107**).

A to je vše...



Nezapomeňte programovat!